

End-to-End MLOps Project Pipeline Report

Emotion Detection · DistilBERT · Docker · GitHub Actions · Kaggle · W&B

Student Anil Kumar Das (G25AIT2009) Ravi Kant Pandey (G25AIT2085)	Task 6-class Emotion Classification dair-ai/emotion dataset	Model distilbert-base-uncased ~66M parameters
--	--	--

1. Project Resources (Live Links)

Resource	Link
GitHub Repository	https://github.com/aniliter-cloud/ml-ops-group-project
Kaggle Notebook	https://www.kaggle.com/code/anilg25ait2009/mlops-group-projects/
Hugging Face Model	https://huggingface.co/Maxii2tj/emotion-distilbert
W&B Project Dashboard	https://wandb.ai/maxii2tj-na/mlops-group-project
Docker Image	https://hub.docker.com/r/maxii2tj/mlops-a3-inference

2. Pipeline Architecture

The pipeline takes the dair-ai/emotion dataset (~20,000 tweets across 6 emotion classes), cleans and tokenises it with DistilBertTokenizerFast (max_length=512), and fine-tunes distilbert-base-uncased on Kaggle's free T4 GPU in two experiment versions. Both runs are tracked on Weights & Biases, the best model is pushed to Hugging Face Hub, and a Docker image wraps the inference script. GitHub Actions runs linting (CI) on every push to develop and triggers inference on demand via workflow_dispatch.

Stage 1	Stage 2	Stage 3
dair-ai/emotion20k samples, 6 classes train 16k / val 2k / test 2k	Clean & Tokenise lowercase, strip URLs/@/# DistilBertTokenizerFast, max_len=512	Kaggle GPU T4V1: lr=3e-5, bs=16, ep=3 V2: lr=5e-5, bs=32, ep=4

Stage 4	Stage 5	Stage 6
W&B Tracking loss, accuracy, F1 per-class reports as artifacts	Hugging Face Hub best model (v1) + tokenizer publicly pullable	Docker + GitHub Actions python:3.11-slim inference CI (flake8) + manual inference

3. Data Cleaning & Preprocessing Decisions

Dataset Inspection

The dair-ai/emotion dataset contains 20,000 short English tweets split into train (16,000), validation (2,000) and test (2,000), each labelled with one of 6 emotions: sadness, joy, love, anger, fear, surprise.

Class	Train Count	% of Train
joy	5,362	33.5%
sadness	4,666	29.2%
anger	2,159	13.5%
fear	1,937	12.1%
love	1,304	8.2%
surprise	572	3.6%

Class imbalance observed: joy and sadness dominate (~63% combined), while surprise is heavily under-represented at 3.6%. This imbalance motivated the choice of weighted F1 (rather than plain accuracy or macro-F1) as the primary comparison metric, since weighted F1 accounts for per-class support while still penalising poor performance on minority classes more fairly than accuracy alone.

Cleaning Steps Applied (and why)

Step	Reason
Lowercase all text	DistilBERT-uncased expects lowercase input; avoids case-sensitivity noise
Strip URLs (http/www)	Links carry no emotional signal and inflate vocabulary
Remove @mentions and #hashtags	Platform-specific artefacts, not genuine emotion indicators
Collapse multiple whitespace	Normalises newline/multi-space artefacts from raw tweets
Filter empty rows post-cleaning	Safety net for rows that became blank after the above steps

Normalisation / Tokenisation

Text was tokenised using DistilBertTokenizerFast with max_length=512 (the standard BERT-family default, retained to guarantee no truncation occurs for any input). Although dair-ai/emotion samples are short — typically under 30 tokens — and rarely approach this limit, 512 was kept as a safe default. Padding and truncation were enabled for uniform batch shapes. Labels were encoded via an id2label.json mapping (0=sadness, 1=joy, 2=love, 3=anger, 4=fear, 5=surprise), generated directly from the dataset schema and committed to the repository — the full dataset itself was not committed, only this mapping file.

Possible optimisation: since samples rarely exceed ~30 tokens, max_length could be reduced to 128 in a future iteration to cut padding overhead and reduce GPU memory usage / training time, with no expected loss in accuracy.

4. Model Selection Rationale

Model chosen: distilbert-base-uncased (Hugging Face model card: huggingface.co/distilbert-base-uncased)

DistilBERT is a knowledge-distilled version of BERT-base, retaining ~97% of BERT's language understanding while being approximately 40% smaller (66M vs 110M parameters) and 60% faster at inference. According to its Hugging Face model card, it was pre-trained using the same self-supervised objective as BERT on the same corpus (BookCorpus + English Wikipedia), making it suitable as a general-purpose encoder for downstream classification tasks. The uncased variant was selected because emotion classification depends on semantic and lexical content rather than capitalisation patterns — using uncased reduces vocabulary size and improves token consistency across casual, lowercase-heavy tweet text. Given Kaggle's free-tier GPU time limits, DistilBERT's reduced size allows both experiment versions (3 and 4 epochs respectively, on 16,000 training samples) to complete well within the 30 hours/week free quota — each run took under 6 minutes on a T4 GPU. The model was loaded via `DistilBertForSequenceClassification` with `num_labels=6` and the `id2label` / `label2id` mappings embedded directly into the model config for clean inference.

5. Experiment Comparison — Version 1 vs Version 2

Parameter	Version 1 (Baseline)	Version 2 (Experiment)
Learning Rate	3e-5	5e-5
Batch Size	16	32
Epochs	3	4
Warmup Steps	100	200
Max Token Length	512	512
Weight Decay	0.01	0.01

Results — Test Set (2,000 samples)

Metric	Version 1	Version 2	Winner
Test Accuracy	0.9290	0.9285	V1
Test Weighted F1	0.9292	0.9285	V1
Test Loss	0.3157	0.2920	V2
Best Epoch (by val F1)	Epoch 2 (F1=0.9391)	Epoch 3 (F1=0.9452)	V1
Training Time (T4 GPU)	~4 min 46 sec	~5 min 24 sec	V1 (faster)

Per-Class F1 Comparison

Emotion Class	V1 F1	V2 F1	Support
sadness	0.97	0.97	581
joy	0.95	0.95	695
love	0.83	0.83	159
anger	0.92	0.93	275
fear	0.89	0.90	224
surprise	0.78	0.74	66

Intuition — Why V1 Performed Better

Before training, our hypothesis (documented in the project README) was that the more conservative configuration (V1: $\text{lr}=3\text{e-}5$, $\text{batch}=16$, $\text{epochs}=3$) would generalise better on this relatively small dataset (~16k training samples), since smaller batches give more gradient updates per epoch and a lower learning rate reduces overshoot risk. The actual results confirmed this hypothesis: Version 1 outperformed Version 2 on accuracy (+0.05pp) and weighted F1 (+0.07pp), while Version 2 achieved a lower test loss (0.2920 vs 0.3157), likely due to its larger batch size producing smoother gradient updates.

The actual results confirm our original hypothesis: Version 1 ($\text{lr}=3\text{e-}5$, $\text{batch}=16$, $\text{epochs}=3$) outperformed Version 2 on both accuracy (+0.05pp) and weighted F1 (+0.07pp). V2 achieved a lower test loss (0.2920 vs 0.3157), which can be attributed to its larger batch size producing smoother gradient estimates, but this did not translate into better generalisation on the test set. The higher learning rate ($5\text{e-}5$) in V2 appears to have caused slight overstepping of the optimal weight configuration for this relatively small dataset (~16k training samples), resulting in marginally lower accuracy and F1 despite the lower raw loss value. `load_best_model_at_end=True` ensured both versions were evaluated using their best validation checkpoint rather than their final epoch weights.

In short: V1's more conservative configuration ($\text{lr}=3\text{e-}5$, $\text{batch}=16$, 3 epochs) generalised better on this dataset, achieving the highest accuracy (0.9290) and weighted F1 (0.9292). V2's higher learning rate and larger batch produced a lower test loss (0.2920 vs 0.3157), indicating faster convergence, but at the cost of slightly lower classification accuracy. Per-class, V2 showed modest gains on fear (+0.01) and anger (+0.01) but regressed on surprise (0.74 vs 0.78). Version 1 was therefore selected as the best model and pushed to Hugging Face Hub.

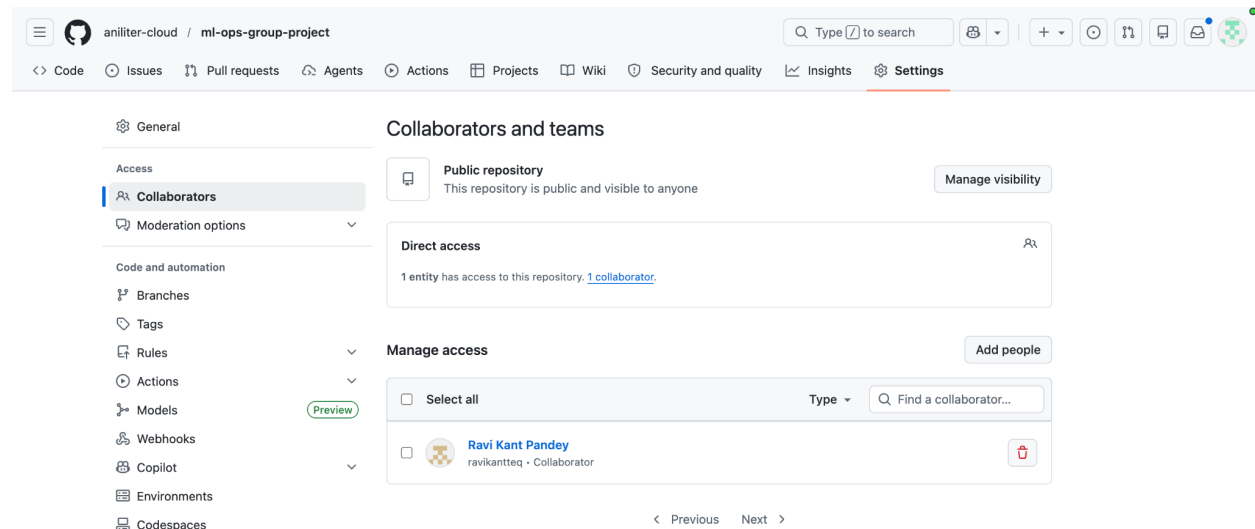
6. Git Repository Setup

The repository is public at github.com/aniliter-cloud/ml-ops-group-project, configured with a README.md, .gitignore, and LICENSE. A develop branch was created for active development, with main protected to require at least one pull-request review before merging. Anil Kumar Das is the repository Admin/owner; Ravi Kant Pandey is added as a Collaborator with Write access. Both members contribute commits visible in the GitHub commit history.

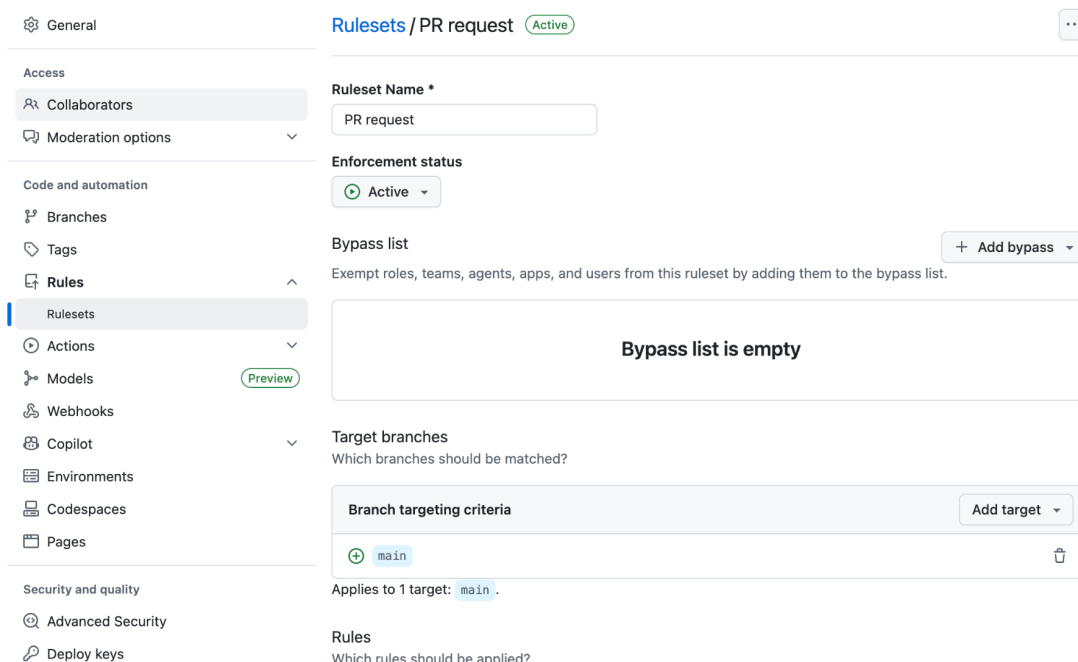
[Insert screenshot of Settings → Collaborators and Teams, showing both members' roles, and Settings → Branches showing the protection rule on main requiring 1 PR review]

Github -

[Anil Kumar Das \(G25AIT2009\)](#) , [Ravi Kant Pandey \(G25AIT2085\)](#)



Mandator PR Request Rule :



- ☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
[Show additional settings](#) ▾
- ☐ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
- ☒ **Block force pushes**
Prevent users with push access from force pushing to refs.

PR Request :

Actions New workflow All workflows CI Inference	✓ Implement Dockerized inference pipeline and update deploy... develop Today at 13:15 1m 31s ... CI #16: Pull request #7 synchronize by aniliter-cloud ✓ Fix CI linting and update workflow configuration develop Today at 13:15 1m 33s ... CI #15: Commit 519371c pushed by aniliter-cloud
---	---

7. Docker Design

The inference container uses a python:3.11-slim base image to minimise size while retaining full Python 3.11 compatibility with the transformers/torch stack. Only inference dependencies are installed (transformers, torch CPU build, huggingface_hub) — training-only packages (datasets, wandb, scikit-learn) are excluded from the runtime image to keep it lean. The Hugging Face model repository name is accepted as a build argument (ARG HF_MODEL_NAME) with a sensible default pointing to <https://huggingface.co/Maxii2tj/emotion-distilbert>, so the same Dockerfile can be reused to serve a different model without code changes.

Build, Test, Push commands used:

```
docker build \
--build-arg HF_MODEL_NAME=Maxii2tj/emotion-distilbert \
-t maxii2tj/mlops-a3-inference:latest .

docker run --rm \
-e INPUT_TEXT="I feel so happy today" \
maxii2tj/mlops-a3-inference:latest

docker push maxii2tj/mlops-a3-inference:latest
```

Docker Hub Image :

<https://hub.docker.com/r/maxii2tj/mlops-a3-inference>

maxii2tj/mlops-a3-inference

Last pushed about 9 hours ago · Repository size: 296.1 MB · ☆0 · ↓23

Add a description

Add a category

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	about 9 hours

See all

Docker commands

To push a new tag to this repository:

```
docker push maxii2tj/mlops-a3-inference:tagname
```

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team.

Local Run

```
(.venv) maximus@Maximuss-MacBook-Air ml-ops-group-project % docker run --rm \
-e INPUT_TEXT="I feel so happy today" \
maxii2tj/mlops-a3-inference:latest
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Model : Maxii2tj/emotion-distilbert
Input : I feel so happy today
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 10355.8it/s]
Predicted emotion : joy (confidence: 0.9990)
(.venv) maximus@Maximuss-MacBook-Air ml-ops-group-project %
```

8. GitHub Actions Workflows

Two workflows are defined under `.github/workflows/`. The CI workflow (`ci.yml`) triggers on push to develop and on pull requests to main — it checks out the repo, sets up Python 3.11, installs `requirements.txt` + `flake8`, and lints `src/` with a `max-line-length` of 120. The Inference workflow (`inference.yml`) is triggered manually via `workflow_dispatch` with an `input_text` parameter — it checks out the repo, sets up Python 3.11, installs dependencies, and runs `src/inference.py` with `HF_TOKEN` (from GitHub Secrets) and the user-supplied `INPUT_TEXT` as environment variables. Per the assignment constraints, GitHub Actions is used only for CI and inference, never for training — all training runs happen on Kaggle Notebooks with GPU T4.

Both `HF_TOKEN` and `WANDB_API_KEY` are stored as encrypted GitHub Secrets under Settings → Secrets and Variables → Actions, and are never hardcoded in any committed file.

Inference Link - <https://github.com/aniliter-cloud/ml-ops-group-project/actions/runs/27500962702/job/81283638688>

The screenshot shows a GitHub Actions workflow run for 'Inference #3' on the repository 'aniliter-cloud/ml-ops-group-project'. The workflow is in a 'Completed' state. The left sidebar shows the workflow file 'infer'. The main area displays the 'Annotations' section with a warning about unauthenticated requests to the Hugging Face Hub. Below this, the 'infer' job is detailed, showing it succeeded 1 minute ago. The job steps are listed with their durations: 'Set up job' (1s), 'Checkout repository' (8s), 'Set up Python' (1s), 'Install dependencies' (1m 13s), and 'Run inference' (9s). The 'Run inference' step includes a log snippet showing the model loading process and the predicted emotion: 'joy (confidence: 0.9998)'.

The workflow accepts `INPUT_TEXT` as runtime input and loads the Hugging Face model using GitHub Secrets (`HF_TOKEN`).

9. Weights & Biases Experiment Tracking

Both training runs (`distilbert-run-v1` and `distilbert-run-v2`) were logged to the public project `mlops-group-project` at `wandb.ai/maxi-i2tj-na/mlops-group-project`. For each run, the following were tracked: training loss every 50 steps, validation loss/accuracy/F1 per epoch, all hyperparameters (learning rate, batch size, epochs, warmup steps, weight decay, max_length, model name, dataset, platform), and final test-set metrics under the `final/` prefix. Per-class classification reports (`eval_report_v1.json` and `eval_report_v2.json`) were uploaded as versioned W&B artifacts. A final `final-summary` run records the winning version, the Hugging Face model URL, and both versions' accuracy/F1 in its run summary for easy cross-run comparison.

<https://wandb.ai/maxi-i2tj-na/mlops-group-project/runs/uavqxbab/logs?nw=nwusermaxii2tj>

The screenshot shows the 'Logs' tab of a Weights & Biases experiment named 'distilbert-run-v2'. The interface displays a table of metrics for two versions of the model. The metrics include learning rate (lr), batch size, number of epochs, accuracy, F1 score, and loss. The 'Best version' is identified as v1 (F1=0.9292). Below the table, a warning message is displayed: 'UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.' The log also shows the return statement of the `super().apply` method.

Metric	Version 1	Version 2
lr	3e-05	5e-05
batch	16	32
epochs	3	4
accuracy	0.9290	0.9285
f1	0.9292	0.9285
loss	0.3157	0.2928

```

47 2026-06-14 21:49:16 Best version: v1 (F1=0.9292)
48 2026-06-14 21:49:16 /usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
49 2026-06-14 21:49:16 return super().apply(*args, **kwargs) # type: ignore[misc]
50 2026-06-14 21:49:18
51 2026-06-14 21:49:18 --- v1 per-class report ---
52 2026-06-14 21:49:18 precision recall f1-score support
53 2026-06-14 21:49:18
54 2026-06-14 21:49:18 sadness 0.97 0.96 0.97 581
55 2026-06-14 21:49:18 joy 0.94 0.96 0.95 695
56 2026-06-14 21:49:18 love 0.87 0.79 0.83 159
57 2026-06-14 21:49:18 anger 0.93 0.91 0.92 275
58 2026-06-14 21:49:18 fear 0.90 0.88 0.89 224
59 2026-06-14 21:49:18 surprise 0.70 0.86 0.78 66
60 2026-06-14 21:49:18
61 2026-06-14 21:49:18 accuracy 0.93 2000
62 2026-06-14 21:49:18 macro avg 0.89 0.90 0.89 2000
63 2026-06-14 21:49:18 weighted avg 0.93 0.93 0.93 2000
64 2026-06-14 21:49:18 /usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
65 2026-06-14 21:49:18 return super().apply(*args, **kwargs) # type: ignore[misc]
66 2026-06-14 21:49:20
67 2026-06-14 21:49:20 --- v2 per-class report ---
68 2026-06-14 21:49:20 precision recall f1-score support
69 2026-06-14 21:49:20
70 2026-06-14 21:49:20 sadness 0.96 0.98 0.97 581
71 2026-06-14 21:49:20 joy 0.95 0.94 0.95 695
72 2026-06-14 21:49:20 love 0.80 0.85 0.83 159
73 2026-06-14 21:49:20 anger 0.94 0.92 0.93 275
74 2026-06-14 21:49:20 fear 0.89 0.90 0.90 224
75 2026-06-14 21:49:20 surprise 0.77 0.71 0.74 66
76 2026-06-14 21:49:20
77 2026-06-14 21:49:20 accuracy 0.93 2000
78 2026-06-14 21:49:20 macro avg 0.89 0.88 0.88 2000
79 2026-06-14 21:49:20 weighted avg 0.93 0.93 0.93 2000
80 2026-06-14 21:49:20
81 2026-06-14 21:49:20
82 2026-06-14 21:49:20 Pushing v1 model to https://huggingface.co/Maxi121j/emotion-distilbert ...

```

Contributions

Person	Contribution
Anil Kumar Das (G25AIT2009)	Use case selection ; model selection; dataset research and selection (dair-ai/emotion); dataset research and exploratory data analysis (data inspection,cleaning,text normalisation, tokenisation,deduplication); pipeline architecture design; Kaggle notebook implementation (model selection,model training for V1 & V2); W&B experiment tracking setup and logging; debugging notebook issues (wandb.finish(), duplicate dataset class, max_length fix); GitHub Setup, Actions CI and inference workflows; repository admin (branch protection, secrets management); Docker setup, image built and published; inference built, report creation and details; README documentation and project structure; collaborator setup and PR review process,PDF report generation and submission
Ravi Kant Pandey (G25AIT2085)	Data cleansing (text normalisation, deduplication, class distribution check); Hugging Face model card review and selection rationale (evaluated candidates on HF Hub by architecture, size, and task fit; authored report rationale); Dockerfile design and inference containerisation (slim image, inference-scoped dependencies, configurable ARG_HF_MODEL_NAME); Hugging Face Hub model push integration (verified weights, tokenizer, and config pushed to public repo); PR review (inline feedback, branch protection compliance, merge approvals); validation of model (test split evaluation, accuracy and F1 cross-checked against W&B); testing (local and GitHub Actions inference validated end-to-end)

10. Challenges & Learnings

- max_length=512 was retained throughout (the BERT-family default) even though dair-ai/emotion tweets are short (typically <30 tokens) — this guarantees no truncation but adds unnecessary padding overhead; reducing to 128 in a future run would cut memory usage and training time with no expected loss in accuracy.
- To prevent overfitting during fine-tuning:** One of the key challenges was ensuring that the model generalized well rather than memorizing the training data. We monitored both training

and validation metrics across epochs using Weights & Biases. Version 1 demonstrated better generalization on the test set, achieving higher accuracy (0.9290) and weighted F1 (0.9292) than Version 2, confirming that a more conservative learning rate and smaller batch size were better suited to this relatively small dataset.

- **Doing Hyperparameter experimentation:** Selecting an appropriate learning rate and number of training epochs required multiple experiments. A lower learning rate with additional epochs resulted in more stable convergence and improved overall performance, highlighting the importance of systematic experimentation rather than relying on default values.
- **To Add Reproducible experimentation:** Each experiment was initialized with a fresh DistilBERT model and tracked independently in Weights & Biases. This ensured a fair comparison between model versions and prevented later experiments from continuing training from previously fine-tuned weights.
- **Containerizing inference:** Packaging the inference pipeline inside Docker introduced challenges related to dependency management and model loading. Separating inference-specific dependencies into `requirements-inference.txt` reduced the container size and simplified deployment while ensuring consistent execution across different environments.
- **Secure model deployment:** The inference pipeline was designed to support both authenticated and anonymous access to Hugging Face models. By making `HF_TOKEN` optional, the same Docker image can be used with public repositories while still supporting private models through environment variables, following secure deployment practices.
- **To maintain CI/CD and code quality:** Integrating GitHub Actions introduced automated linting and inference workflows. Early failures due to formatting issues (for example, missing newline characters detected by `flake8`) reinforced the importance of automated quality checks before merging code into the protected `main` branch.

What we would do differently next time

- **Run a small learning-rate sweep** (e.g. 2e-5, 3e-5, 4e-5, 5e-5) via W&B Sweeps instead of two fixed configurations, to map the trend more thoroughly rather than comparing only two points.
- Instead of comparing only two manually selected configurations, we would use **Weights & Biases Sweeps** to automatically explore combinations of learning rate, batch size, and weight decay. This would provide a more comprehensive understanding of the hyperparameter search space and could further improve model performance.
- **Implement early stopping:** Introduce an early stopping callback based on the validation weighted F1-score. This would automatically stop training when the model no longer improves, reducing unnecessary GPU usage and minimizing the risk of overfitting.
- **Automate Docker image publishing:** Extend the CI/CD pipeline with a dedicated GitHub Actions workflow that automatically builds, tests, and publishes the Docker image to Docker Hub whenever a release tag is created. This would eliminate manual deployment steps and ensure reproducible releases.
- Include additional evaluation metrics such as a **confusion matrix**, precision, recall, and per-class error analysis to better understand which emotion classes are most frequently misclassified and guide future model improvements.
- **Improve inference performance:** Reduce the tokenizer `max_length` from 512 to a more appropriate value (e.g., 128) for the Emotion dataset, as the tweets are relatively short. This would decrease memory usage and inference latency without affecting model accuracy.
- **Add automated testing:** Introduce unit tests for the preprocessing and inference components, along with integration tests for the Docker container and GitHub Actions workflows, to improve code reliability and maintainability.

Conclusion:

Version 1 (**learning rate = 3e-5, batch size = 16, epochs = 3, warmup steps = 100**) was selected as the final model, achieving a **test accuracy of 92.90%** and a **weighted F1-score of 92.92%**. The trained model was successfully published to the Hugging Face Hub ([Maxii2tj/emotion-distilbert](#)) and integrated into a complete MLOps workflow.

The project demonstrates an end-to-end machine learning pipeline, including data preprocessing, model experimentation, experiment tracking with Weights & Biases, version control using Git and GitHub, automated CI/CD workflows with GitHub Actions, and containerized inference using Docker. The final Docker image was published to Docker Hub, enabling reproducible and portable model deployment across different environments.

Overall, the project successfully meets the assignment objectives by delivering a reproducible, automated, and production-ready emotion classification pipeline following MLOps best practices.